

SIMATS ENGINEERING

ASSESSMENT TOOL-1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 20

Duration: 30 Minutes
Total Marks:20

Annexure A

MCQ Test

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

UBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	30%	Demonstrates thorough understanding; answers all questions accurately	Shows good understanding with few errors	Partial understanding; several incorrect responses	Lacks understanding; majority of answers incorrect
Accuracy of Answers	25%	All answers are correct	Most answers are correct with minor mistakes	Some correct answers; frequent errors	Mostly incorrect answers
Application of Knowledge	20%	Correctly applies concepts to problem-based questions	Applies concepts with minor errors	Limited ability to apply concepts	Unable to apply concepts
Time Management	15%	Completes quiz well within time efficiently	Completes on time with minor delays	Requires extra time or rushes through questions	Unable to complete within given time
Question Interpretation	10%	Interprets all questions correctly and responds appropriately	Misinterprets a few questions	Often misinterprets questions	Unable to understand questions



CO2_AT1_MCQ TEST

Total points 20/20

✓ In informed search strategies, some algorithms use only the heuristic value 1/1 to decide which node to expand next. This approach focuses on selecting the node that appears closest to the goal without considering the cost already incurred.

Which algorithm follows this principle?

- ☐ Breadth First Search
- ☐ Uniform Cost Search
- ☒ Greedy Best-First Search ✓
- ☐ Depth First Search

✓ A* search is widely used in pathfinding and graph traversal because it 1/1 combines the actual cost from the start node and the estimated cost to the goal. This balance allows it to find optimal solutions efficiently when the heuristic is admissible.

What is the evaluation function used in A* search?

- ☐ $f(n) = g(n)$
- ☐ $f(n) = h(n)$
- ☒ $f(n) = g(n) + h(n)$ ✓
- ☐ $f(n) = g(n) \times h(n)$

✓ Heuristic functions play a crucial role in informed search by guiding the 1/1 algorithm toward the goal. A good heuristic should not overestimate the actual cost and should improve search efficiency.

What property ensures optimality in A* search?

- ☐ Completeness
- ☒ Admissibility ✓
- ☐ Depth limit
- ☐ Backtracking

✓ Local search algorithms operate by starting with an initial solution and 1/1 iteratively improving it based on a given objective function. These methods are commonly used in optimization problems where the path is not important but the final solution is.

Which of the following is a local search algorithm?

- ☐ A* Search
- ☒ Hill Climbing ✓
- ☐ Breadth First Search
- ☐ Uniform Cost Search

✓ In adversarial search, agents compete against each other in a game 1/1 environment. The decision-making process involves considering both the agent's moves and the opponent's possible responses.

Which algorithm is commonly used for such decision-making?

- ☐ Greedy Search
- ☒ Minimax Algorithm ✓



✓ Local search algorithms operate by starting with an initial solution and iteratively improving it based on a given objective function. These methods are commonly used in optimization problems where the path is not important but the final solution is. 1/1
Which of the following is a local search algorithm?

- ☐ A* Search
- ☒ Hill Climbing ✓
- ☐ Breadth First Search
- ☐ Uniform Cost Search

✓ In adversarial search, agents compete against each other in a game environment. The decision-making process involves considering both the agent's moves and the opponent's possible responses. 1/1
Which algorithm is commonly used for such decision-making?

- ☐ Greedy Search
- ☒ Minimax Algorithm ✓
- ☐ Backtracking
- ☐ Uniform Search

✓ A delivery robot in a warehouse selects paths based only on the estimated distance to the destination without considering obstacles already encountered. This sometimes leads to suboptimal paths but faster decisions. 1/1
Which algorithm is the robot using?

- ☐ A* Search
- ☒ Greedy Best-First Search ✓
- ☐ Uniform Cost Search
- ☐ Depth Limited Search

✓ A GPS navigation system calculates routes by combining the distance already travelled and an estimate of the remaining distance to the destination. This helps in finding the shortest path efficiently. 1/1
Which algorithm best represents this approach?

- ☐ Depth First Search
- ☒ A* Search ✓
- ☐ Hill Climbing
- ☐ Backtracking

✓ A university is scheduling exams for multiple subjects with constraints such as no overlapping exams and limited rooms. The solution must satisfy all constraints while assigning time slots. 1/1
This problem is best modeled as:

- ☐ Game Playing Problem
- ☒ Constraint Satisfaction Problem ✓
- ☐ Heuristic Search Problem

☐ Backtracking

✓ A university is scheduling exams for multiple subjects with constraints such as no overlapping exams and limited rooms. The solution must satisfy all constraints while assigning time slots.
This problem is best modeled as:

☐ Game Playing Problem

☒ Constraint Satisfaction Problem ✓

☐ Heuristic Search Problem

☐ Optimization Problem

✓ An AI-based chess program evaluates all possible moves and anticipates the opponent's responses before making a decision. It assigns utility values to game states to determine the best move.
Which algorithm is being applied here?

☐ Greedy Search

☐ Hill Climbing

☒ Minimax Algorithm ✓

☐ Breadth First Search

✓ A robot explores an unknown environment where it does not have prior knowledge of obstacles. It makes decisions step-by-step and updates its knowledge as it moves.
Which type of agent is suitable for this scenario?

☐ Offline Agent

☒ Online Search Agent ✓

☐ Static Agent

☐ Deterministic Agent

✓ A* search guarantees optimal solutions only when certain conditions are satisfied by the heuristic function. If these conditions are violated, the algorithm may still find a solution but not necessarily the best one.
Why is an admissible heuristic important?

☐ It reduces memory usage

☒ It ensures optimality ✓

☐ It increases branching factor

☐ It avoids loops

✓ In large game trees, evaluating every node becomes computationally expensive. Alpha-Beta pruning helps eliminate branches that do not affect the final decision, thereby improving efficiency.
What is the primary benefit of Alpha-Beta pruning?

☐ Reduces depth of tree

☒ Reduces number of nodes evaluated ✓

☐ Changes game rules

☐ Improves heuristic accuracy

✓ Hill Climbing algorithm continuously moves toward better states based on local improvements. However, it may fail to reach the global optimum due to certain limitations in the search space.
What is the main drawback of Hill Climbing?

- ☐ Changes game rules
- ☐ Improves heuristic accuracy

✓ Hill Climbing algorithm continuously moves toward better states based on local improvements. However, it may fail to reach the global optimum due to certain limitations in the search space.
What is the main drawback of Hill Climbing?

- ☐ High memory usage
- ☐ Revisiting nodes
- ☒ Getting stuck in local maxima ✓
- ☐ Requires complete tree

✓ Constraint Satisfaction Problems involve variables, domains, and constraints. A valid solution must assign values to all variables such that no constraints are violated.
What ensures a valid CSP solution?

- ☐ Heuristic function
- ☐ Path cost
- ☒ Constraint satisfaction ✓
- ☐ Depth-first traversal

✓ In adversarial games, evaluation functions are used when it is not feasible to explore the entire game tree. These functions estimate the desirability of a game state.
What is the role of an evaluation function?

- ☐ Generate successors
- ☒ Estimate utility of a state ✓
- ☐ Reduce branching factor
- ☐ Store moves

✓ In A* search, the evaluation function is calculated using both the path cost and heuristic value. Suppose a node has a path cost of 7 and a heuristic estimate of 5.
What is the value of $f(n)$?

- ☒ 12 ✓
- ☐ 2
- ☐ 35
- ☐ 7

✓ A heuristic function sometimes overestimates the actual cost to reach the goal. This violates the admissibility condition required for optimal solutions in A* search.
What happens in such a case?

- ☐ A* remains optimal
- ☒ A* becomes non-optimal ✓
- ☐ A* becomes BFS
- ☐ A* stops working

✓ Consider a search tree with a branching factor of 2 and depth of 4. The total number of nodes generated in a full binary tree can be calculated analytically.
What is the approximate number of nodes?



☐ Store moves

✓ In A* search, the evaluation function is calculated using both the path cost 1/1 and heuristic value. Suppose a node has a path cost of 7 and a heuristic estimate of 5.
What is the value of $f(n)$?

☒ 12 ✓

☐ 2

☐ 35

☐ 7

✓ A heuristic function sometimes overestimates the actual cost to reach the 1/1 goal. This violates the admissibility condition required for optimal solutions in A* search.
What happens in such a case?

☐ A* remains optimal

☒ A* becomes non-optimal ✓

☐ A* becomes BFS

☐ A* stops working

✓ Consider a search tree with a branching factor of 2 and depth of 4. The 1/1 total number of nodes generated in a full binary tree can be calculated analytically.
What is the approximate number of nodes?

☐ 10

☐ 15

☒ 31 ✓

☐ 16

✓ In a minimax tree, each node has 3 children and the depth of the tree is 3. 1/1 The total number of leaf nodes depends on the branching factor raised to the depth.
How many leaf nodes are present?

☐ 9

☒ 27 ✓

☐ 6

☐ 12

✓ Alpha-Beta pruning improves the efficiency of minimax by reducing the 1/1 number of nodes evaluated. In the best-case scenario, it significantly lowers time complexity.
What is the best-case time complexity?

☐ $O(b^d)$

☒ $O(b^{(d/2)})$ ✓

☐ $O(d^b)$

☐ $O(\log b)$